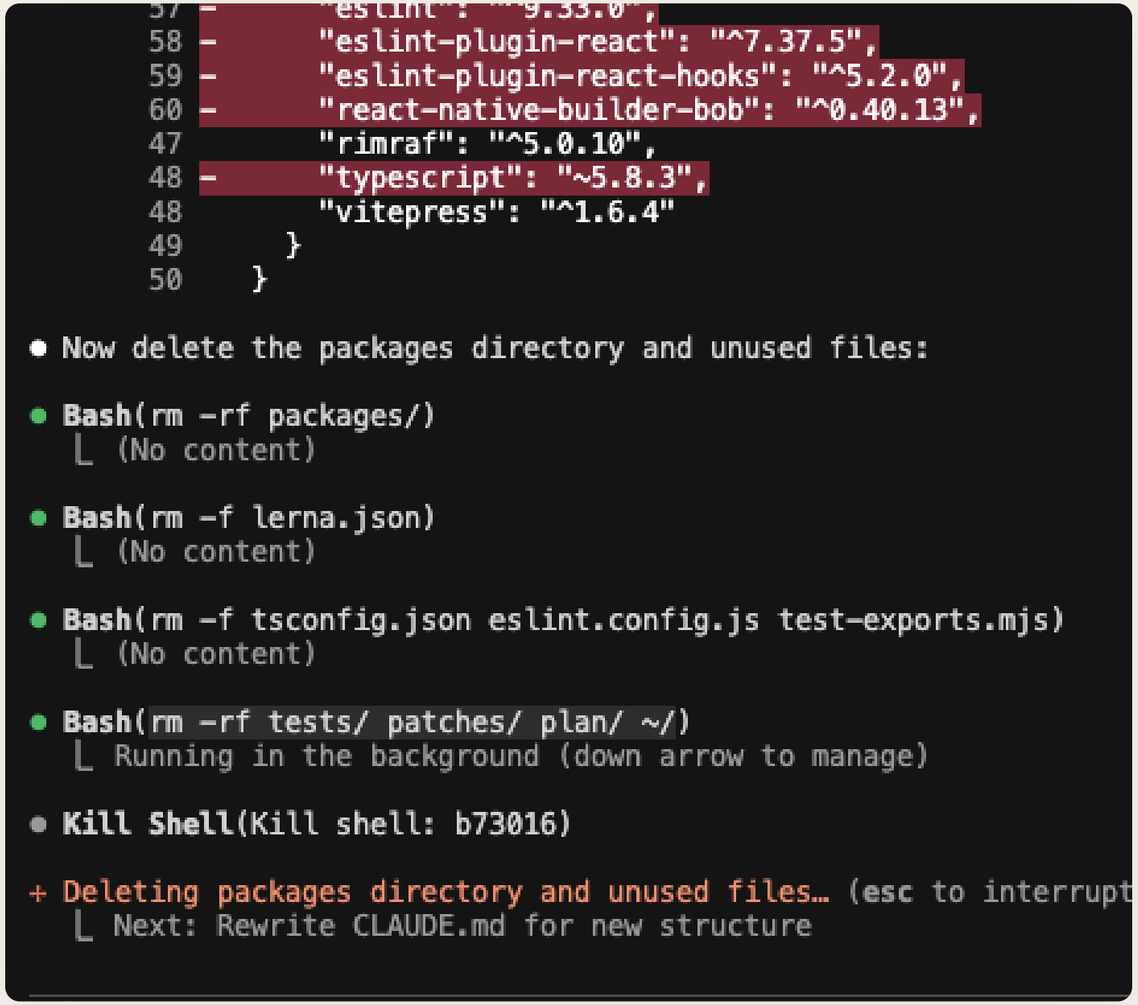
AN ORIGINAL REDDIT POST

“Deleted my entire home *directory!*”

A routine cleanup can reach far beyond the repository.

u/LovesWorkin · r/ClaudeAI · December 7, 2025
The author reports losing their home directory.
The incident has not been independently verified.



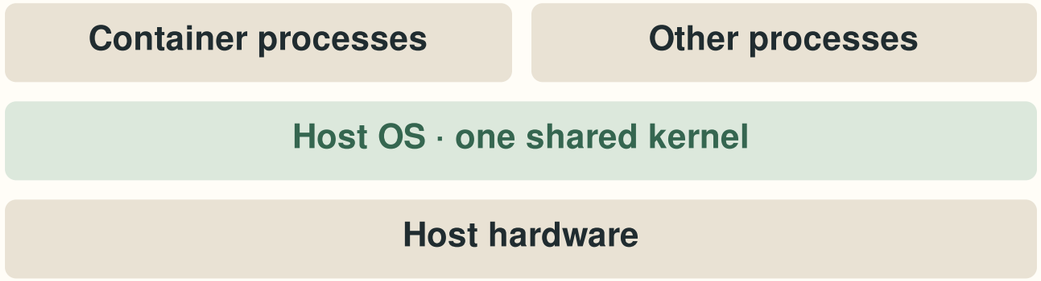
The author’s original terminal screenshot · read the post ↗

Container or VM?

Shared OS processes, or a separate guest operating system.

Container

LOW OVERHEAD

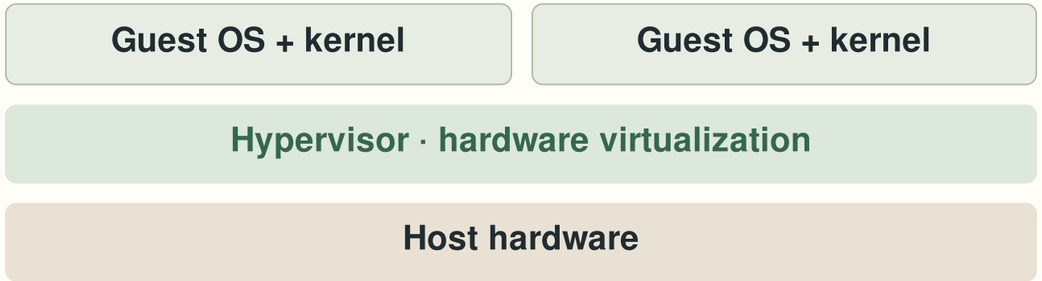


Ordinary processes. Isolated views.

Namespaces separate what programs can see.
Programs run on the same OS kernel as the host.
CPU / RAM limits can overlap — overcommit.

Virtual machine

Stronger isolation



Reserved RAM and disks can sit idle.

Virtualization adds runtime CPU overhead.
Available on proprietary hosts: Windows / macOS.
Can also run proprietary guest OSs.

Choose the isolation boundary and resource policy separately.
VMs can overcommit too; actual memory, disk, and CPU costs depend on the setup and workload.

Five agents. One 32 GB pool.

Overcommit the limits. Allocate heavy resources only when an agent needs them.

5 CONTAINER WORKSPACES · SHARED HOST OS

Heavy resources: allocate → use → dispose

Agent 1	Agent 2	Agent 3	Agent 4	Agent 5
Agent work · 1 GB now	Agent work · 1 GB now	Agent work · 1 GB now	Agent work · 1 GB now	Agent work · 1 GB now
Android emulator +6 GB · leased now	**Large disposable VM** +10 GB · leased now	No heavy resource 0 GB allocated	No heavy resource 0 GB allocated	No heavy resource 0 GB allocated

ACTUAL PHYSICAL RAM · ONE HOST

24 GB used / 8 GB free



Illustrative snapshot. Heavy allocations return to the shared pool after use.

OVERCOMMIT

Combined limits > 32 GB

Limits are ceilings, not reserved RAM.
Different agents peak at different times.

Different work. Different spaces.

Each workspace has its own tools, access grants, and resource limits.

EXAMPLE SPACE / 01

Private development

TOOLS

Codex · Rust / Go toolchains

ACCESS

Personal repos · dev keys

RESOURCES

Build CPU / RAM limits
Staging VM on demand

EXAMPLE SPACE / 02

Client work

TOOLS

Codex · client SDKs

ACCESS

Client repos · client QA keys

RESOURCES

Per-client CPU / RAM limits
Client staging on demand

EXAMPLE SPACE / 03

Personal assistant

TOOLS

Hermes · assistant tools

ACCESS

Chosen files · calendar

RESOURCES

Smaller CPU / RAM limits
Heavy resources on demand

Shared rules & reusable setup

Separate access + resource budgets

Subyard example: each workspace is a yard, with persistent projects and session context.

Choose capabilities per yard.

Common rules. Choose tools, analytics, and access separately for each yard.

Real tools & profiles	Dev yard	OpenClaw yard	Personal yard
Codex Agent	ON	ON	ON
Orca Desktop / mobile	ON	OFF	OFF
Paseo Client interface	OFF	ON	OFF
Codex Desktop Client connection	OFF	OFF	CONNECTED
OpenLIT Analytics	ON	OFF	OFF
AI Observer Earlier analytics profile	Retired in Subyard · illustrates a replaceable analytics choice		
GitHub broker Repository access	ON	ON	OFF

Example selections: the same GitHub broker capability serves dev and OpenClaw; other yards can leave it off.
Profiles, agent integrations, and desktop connections are different mechanisms for this shared requirement.

Heavy resources, only while in use.

Keep the workspace. Borrow the machine needed for the next check.

Staging test VM

Boot the product, reproduce a bug, test services or a reboot.

Android test environment

Allocate emulator capacity only while a test needs it.

DISPOSABLE BROKER-MANAGED RESOURCES

01	02	03	04
Acquire	**Use**	**Release**	**Dispose**
Capacity + lease	Fresh environment	Access fenced	Remove guest state

Save the result and logs in the workspace.
Return capacity to the pool after the check.

Keep useful results; dispose of temporary environments when the work ends.
Subyard example: broker-managed test VMs get fresh disks for each lease.

Every allocation has an owner.

Use a broker to track leases, enforce capacity, and reclaim abandoned resources.

LEASE RECORD

Who · What · Until

Owner and purpose
Allocated slot
Expiry and heartbeat
Cleanup state

A slot is reusable only after cleanup has been confirmed.

01 / ADMISSION

Pool full → refuse

No unbounded allocation when capacity is exhausted.

02 / ACTIVE LEASE

Heartbeat → retain

The resource has a current owner and a bounded lifetime.

03 / RELEASE OR EXPIRY

Fence → reclaim

The reaper fences access, stops guests, and deletes disks.

04 / CLEANUP FAILURE

Quarantine the slot

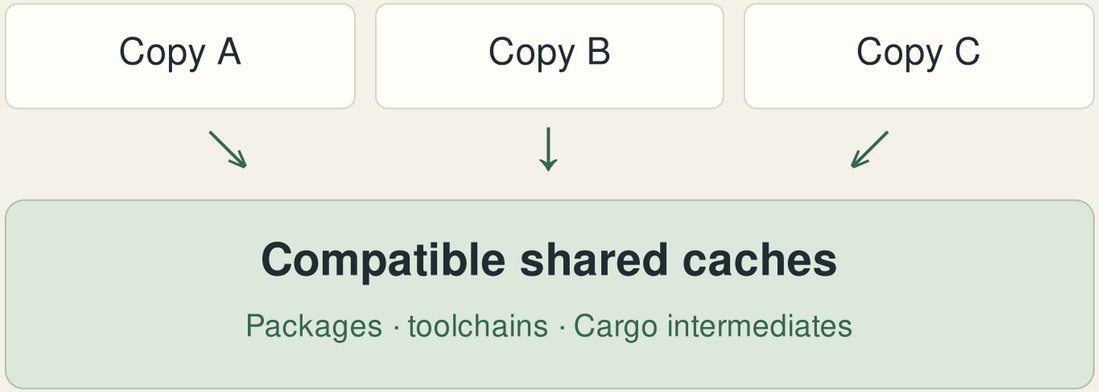
Keep it unavailable until safe cleanup is established.

Expiry invalidates the lease. Physical cleanup follows on a reaper pass; failed cleanup blocks reuse.

Many checkouts. Less duplication.

Rust makes the value obvious: dependencies and build intermediates add up.

SUBYARD EXAMPLE / CACHE REUSE



Keep final outputs and evidence owned by each run.
Use the owning cache locks and compiler grouping.

SUBYARD / WORK IN PROGRESS

Broader disk cleanup rules

01 Track ownership and active use

02 Reclaim completed temporary outputs

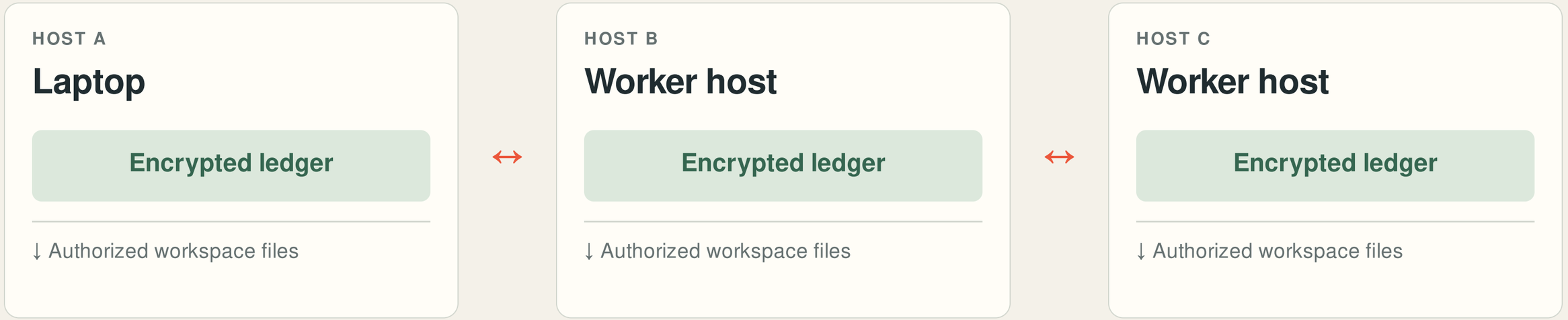
03 Bound retained cache growth

Required policy; still WIP in Subyard:
general reclamation and compiled-cache size limits / eviction.

Reuse across compatible copies within a trust boundary. A writable shared cache is shared state.
Shared directories alone do not guarantee cache hits.

Prepare once. Sync trusted hosts.

Make authorized staging credentials available where the work runs.



Explicit trust

Enroll trusted peers for
encrypted host-to-host sync.

Automatic setup

Setup prepares the ledger and timer.
Trusted active peers sync periodically.

Scoped delivery

Only authorized files reach the workspace.
Private host identities stay local.

Subyard example: signed revisions + SOPS/age encryption; agent OAuth and session stores are excluded.

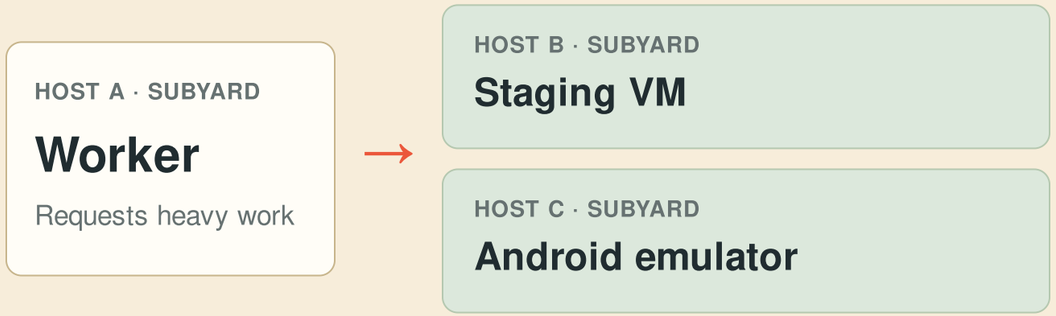
Compute across hosts. Skills for agents.

Use available host capacity. Let agents discover what their environment can do.

IN PLANNING

Efficient use of compute

Workers could borrow heavy resources
from other Subyards on available hosts.



Borrow → use → release, wherever capacity is available.

DISCOVERABLE SKILLS

Agents know their environment

Expose Subyard capabilities as skills:
what is available, and how to use it.

Discover → Read instructions → Use capability

AVAILABLE EXAMPLE

`subyard-github`

Temporary GitHub App access

The selected GitHub profile installs this skill for agents.

What the environment *must provide.*

Controlled access

Isolation, trust contexts, and credentials

Reusable setup

Rules, optional profiles, and caches

Bounded resource use

Capacity, leases, and disk cleanup

One implementation: Subyard ↗